

LM Studio Chat SPA

Single-Page Chat Application with Tool Calling for Local LLM Inference

AUTHOR

Vasileios Kantartzis

EMAIL

vasileios.kantartzis@aptiv.com

DATE

May 7, 2026

Overview

A lightweight, browser-based Single Page Application (SPA) that connects to a local LM Studio server for real-time chat with locally-hosted Large Language Models. The application features a dark-themed UI inspired by LM Studio's native interface, with streaming responses, conversation management, reasoning model support, and an intelligent tool-calling system that enables the model to perform real-world actions.

Architecture

Technology Stack

COMPONENT	TECHNOLOGY	PURPOSE
Frontend	Vanilla HTML5 / CSS3 / JavaScript	Zero-dependency SPA
Proxy Server	Node.js (built-in http module)	CORS proxy + static file server + tool executor
Backend API	LM Studio Local Server	OpenAI-compatible REST API
Storage	Browser localStorage	Conversation & settings persistence

Request Flow

The SPA runs entirely in the browser. A Node.js proxy server bridges the browser-to-API communication gap (CORS), forwarding all `/v1/*` requests to the LM Studio server on port 1234. Streaming responses are piped directly to the browser for real-time rendering. When the model requests tool calls, the proxy intercepts the response, executes tools server-side, feeds results back to the model (up to 5 rounds), then streams the final answer to the client.

Tool-Calling Architecture

LM Studio Chat SPA • Vasileios Kantartzis • May 2026

How it works: The proxy automatically injects tool definitions into every chat request. When the model returns tool calls, the proxy executes them against server-side implementations, appends results to the conversation context, and re-queries the model until it produces a final text response. All of this is transparent to the client, which receives real-time SSE events for tool start/result status.

Project Structure

```
lmstudio-chat-spa/  "€"€ index.html # Main SPA (HTML + CSS + JS, ~1200 lines)  "€"€
server.js # Node.js proxy + static server + tool executor  ""€"€ docs/  "€"€ pdf-
source.html # Documentation source (this file)  ""€"€ project-documentation.pdf # Rendered
PDF
```

Detected Models

MODEL	PARAMETERS	ARCHITECTURE	SIZE	STATUS
qwen/qwen3.6-27b	27B	qwen3.5	17.48 GB	LOADED
qwen/qwen3.5-35b-a3b	35B-A3B	qwen3.5-moe	22.07 GB	Local
text-embedding-nomic-embed-text-v1.5	—	Nomic BERT	84.11 MB	Local

Core Features

Streaming Responses

Real-time token-by-token streaming with incremental markdown rendering

Reasoning Support

Collapsible "Thinking" blocks for reasoning models (e.g., Qwen)

Conversation History

Sidebar with persistent conversations saved to localStorage

Model Switching

Dynamic model selector populated from LM Studio's model list

Generation Settings

Configurable temperature, max tokens, top-p, and system prompt

Stop Generation

Abort button to cancel streaming mid-response

Markdown Rendering

Quick Suggestions

Welcome screen with clickable prompt suggestions

Built-in renderer for code blocks, lists, bold, italic, headings, blockquotes

Tool-Calling System

The proxy server implements 11 tools that the model can invoke automatically. Tools are executed server-side for security and access to system resources.

Available Tools

TOOL	DESCRIPTION	PARAMETERS
🔍 search_web	Search the internet via DuckDuckGo for current information, news, prices, etc.	query (string)
🕒 get_current_time	Get the current date and time in any timezone	timezone (string, default: UTC)
🧮 calculator	Evaluate mathematical expressions (supports sqrt, sin, cos, log, pi, e, ^)	expression (string)
🌐 fetch_url	Fetch and extract text content from any URL (10s timeout, strips scripts/styles)	url (string)
📁 list_files	List files and directories in a local path	directory (string, optional)
📖 read_file	Read a local file (max 1MB, content truncated at 50KB)	filepath (string)
✍️ write_file	Write content to a local file (creates parent dirs automatically)	filepath, content (strings)
🚀 run_command	Execute a shell command (30s timeout, safety-blocked dangerous commands)	command (string), timeout (number)
🌤️ get_weather	Get weather for a location (OpenWeatherMap API or wttr.in fallback)	location (string), units (metric/imperial)
🗣️ text_to_speech_info	Query available TTS models from LM Studio	action (list_models/get_info)
📄 summarize_text	Summarize long text by extracting key sentences up to max word count	text (string), max_length (number)

LM Studio Chat SPA • Vasileios Kantartzis • May 2026

Tool-Calling Loop

The proxy supports up to **5 automated tool-calling rounds** per request. After the model's first response, if tool calls are detected:

- 1. Proxy executes each tool and collects results
- 2. Results are appended to the conversation context
- 3. Proxy re-queries the model with updated context
- 4. Repeats until the model produces a final text response or 5 rounds elapse

Client-Side Tool Events

The client receives real-time Server-Sent Events for tool execution:

EVENT TYPE	WHEN SENT	CLIENT BEHAVIOR
tool_calls_start	Before executing a batch of tool calls	Shows "Running N tools..." indicator
tool_start	Before each individual tool execution	Shows tool name with "running" status badge
tool_result	After each tool completes	Updates badge to "done", stores result

Security

Command Safety

The run_command tool blocks dangerous operations:

Blocked patterns: format, mkfs, rm -rf, fork bombs, dd if=, shutdown, reboot, fdisk

All commands have a 30-second default timeout. Output is truncated at 10,000 characters (stdout) and 5,000 characters (stderr).

File Safety

File read operations are limited to 1MB files with content truncated at 50KB. Write operations create parent directories automatically but operate on resolved absolute paths.

Configuration

Default Settings

PARAMETER	DEFAULT	RANGE
Temperature	0.7	0.0 – 2.0

Max Tokens	4096	64 – 8192
Top P	0.9	0.0 – 1.0
System Prompt	Tool-aware prompt (see below)	Customizable

Default System Prompt

```
You are a helpful assistant. Use available tools when they help answer the user's question.
For current information, use search_web. For math, use calculator. For time questions, use
get_current_time. For weather, use get_weather. For file operations, use read_file,
write_file, or list_files. For system commands, use run_command.
```

Environment Variables

VARIABLE	PURPOSE	DEFAULT
WEATHER_API_KEY	OpenWeatherMap API key for weather tool	Fallback to wttr.in

Getting Started

Prerequisites

- LM Studio installed and running (lms server start)
- Node.js v18+ available
- At least one LLM model loaded

Run Commands

```
# 1. Start LM Studio server lms server start # 2. Start the SPA proxy server cd lmstudio-chat-
spa node server.js # 3. Open in browser # http://localhost:3001
```

API Compatibility

The proxy server is compatible with the OpenAI API specification, supporting:

- GET /v1/models — List available models
- POST /v1/chat/completions — Chat completions (with streaming)

Tool definitions are auto-injected into chat requests. Clients need not specify tools manually.

LM Studio Chat SPA • Vasileios Kantartzis • May 2026